Module 02: Agents in Robotics — Control Agent Design

Learning Objectives

1. Define the **control agent** as the computational entity that maintains state beliefs and computes control actions in a robotic system.
2. Analyze how agent design choices (centralized vs. distributed, model-based vs. model-free) affect control performance and robustness.
3. Apply agent design principles to partition control responsibilities across software components.

Introduction

The control agent is the "mind" of the robotic control-estimation system — the software entity that receives sensor observations, maintains and updates state beliefs, evaluates available actions, and selects the output command. How this agent is structured determines the robot's capabilities, limitations, and failure modes.

Key Concepts

1. Centralized vs. Distributed Control Agents

Robotic systems can organize their control agency in different ways:

- **Centralized agent**: A single control loop receives all sensor data, maintains a unified state estimate, and computes all actuator commands. Simple to implement but creates a single point of failure and may struggle with latency in large systems.
- **Distributed agents**: Multiple control loops each manage a subset of the system. A 6-DOF arm might have 6 independent joint controllers, each minimizing local proprioceptive prediction error. Coordination between joints is handled through dynamic coupling rather than explicit centralized computation.

- **Hierarchical agents**: A cascade structure where high-level agents (trajectory planner) set references for mid-level agents (velocity controller), which set references for low-level agents (current controller). Each level operates at a different timescale and precision.

2. Model-Based vs. Model-Free Agents

Control agents differ in whether they maintain an explicit generative model:

- **Model-based agents**: Maintain an explicit B matrix (dynamics model) and use it for prediction and control. Can anticipate and compensate for future disturbances. Require accurate models.
- **Model-free agents**: Learn direct sensor-to-action mappings without an explicit model. More robust to modeling errors but cannot anticipate — they only react. PID controllers are partially model-free (they don't predict future states).
- **Hybrid agents**: Use a model for prediction but augment with model-free corrections for unmodeled dynamics — combining the anticipatory capacity of model-based methods with the robustness of model-free methods.

3. State Space Design

The control agent's effectiveness depends on choosing the right **state space** — which variables to track:

- **Under-instrumented**: Important states are not tracked (e.g., ignoring joint flexibility in a lightweight arm → oscillation)
- **Over-instrumented**: Too many states increase computational cost without improving control (e.g., tracking every bolt on a rigid frame)
- **Appropriate state space**: Includes all controllable and observable states that affect performance, and excludes states that are neither observable nor relevant

4. Observer Design

When some states are not directly measured (hidden states), the control agent uses an **observer** to estimate them:

- **Full-state observer** (Luenberger observer): Estimates all states from available measurements using the system model
- **Reduced-order observer**: Estimates only the unmeasured states, using measured states directly
- **Disturbance observer**: Estimates unmodeled external disturbances (friction, wind, payload variation) from the discrepancy between predicted and measured behavior — directly analogous to Active Inference surprise detection

Applications

- **Humanoid robot balance controller**: A hierarchical control agent manages humanoid balance — a high-level center-of-mass planner sets posture goals, a mid-level whole-body controller distributes joint targets, and low-level joint controllers track the targets. The hierarchy implements multi-timescale Active Inference with information flowing up (observations) and down (reference signals).
- **Quadrotor with wind disturbance observer**: A quadrotor's control agent includes a disturbance observer that estimates wind gusts from the difference between predicted and actual drone motion. The estimated wind is fed forward into the controller, enabling proactive compensation rather than purely reactive correction.

Conclusion

Control agent design — centralized vs. distributed, model-based vs. model-free, state space and observer design — determines the robotic system's inference and control capabilities. The next module examines perception in the control-estimation framework.